OOP Lab-I: Week 11

Files and Streams – II | Code Solutions

Question 1: Employee Database and Salary Slip

Stores employee details with an automatically generated ID and finds an employee by ID to print a salary slip. Assumed formulas: DA = 10% of basic salary, HRA = 20% of basic salary, and income tax = 10% of gross pay. The lab manual does not specify the percentages; replace them if your instructor provides different rates.

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

const char FILE_NAME[] = "employees.dat";

struct Employee {
    int id;
    char name[50];
    char department[30];
    char designation[30];
    double basic;
};

int generateID() {
    ifstream fin(FILE_NAME, ios::binary);
    Employee e;
    int id = 1000;

    while (fin.read((char*)&e, sizeof(e))) {
        if (e.id > id)
            id = e.id;
    }

    return id + 1;
}

void addEmployee() {
    Employee e;
    e.id = generateID();

    cout << "Enter employee name: ";
    cin >> ws;
    cin.getline(e.name, 50);

    cout << "Enter department: ";
    cin.getline(e.department, 30);

    cout << "Enter designation: ";
    cin.getline(e.designation, 30);

    cout << "Enter basic salary: ";
    cin >> e.basic;

    if (!cin || e.basic < 0) {
        cout << "Invalid input.\n";
        cin.clear();
        cin.ignore(10000, '\n');
        return;
    }

    ofstream fout(FILE_NAME, ios::binary | ios::app);

    if (!fout) {
        cout << "File opening error.\n";
        return;
    }

    fout.write((char*)&e, sizeof(e));
    fout.close();

    cout << "Employee added. ID: " << e.id << '\n';
}

void salarySlip() {
    int id;
```

```
    cout << "Enter employee ID: ";
    cin >> id;

    ifstream fin(FILE_NAME, ios::binary);

    if (!fin) {
        cout << "Employee file not found.\n";
        return;
    }

    Employee e;
    bool found = false;

    while (fin.read((char*)&e, sizeof(e))) {
        if (e.id == id) {
            found = true;
            break;
        }
    }

    fin.close();

    if (!found) {
        cout << "Employee not found.\n";
        return;
    }

    double da = e.basic * 0.10;
    double hra = e.basic * 0.20;
    double gross = e.basic + da + hra;
    double tax = gross * 0.10;
    double net = gross - tax;

    cout << fixed << setprecision(2);
    cout << "\n------ SALARY SLIP ------\n";
    cout << "Employee ID : " << e.id << '\n';
    cout << "Name        : " << e.name << '\n';
    cout << "Department  : " << e.department << '\n';
    cout << "Designation : " << e.designation << '\n';
    cout << "Basic Salary: Rs. " << e.basic << '\n';
    cout << "DA          : Rs. " << da << '\n';
    cout << "HRA         : Rs. " << hra << '\n';
    cout << "Gross Pay   : Rs. " << gross << '\n';
    cout << "Income Tax  : Rs. " << tax << '\n';
    cout << "Net Pay     : Rs. " << net << '\n';
    cout << "-------------------------\n";
}

int main() {
    int choice;

    do {
        cout << "\n1. Add Employee";
        cout << "\n2. Generate Salary Slip";
        cout << "\n3. Exit";
        cout << "\nEnter choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addEmployee(); break;
            case 2: salarySlip(); break;
            case 3: cout << "Exiting.\n"; break;
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 3);

    return 0;
}
```

Question 2: Item Database — Add and Modify

Stores item name, automatically generated item code, unit price, and quantity. The menu allows adding an item or modifying an existing item. Fixed-size binary records and seekp() are used to overwrite the selected record.

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

const char FILE_NAME[] = "items.dat";

struct Item {
    int code;
    char name[50];
    double price;
    int quantity;
};

int generateCode() {
    ifstream fin(FILE_NAME, ios::binary);
    Item item;
    int code = 2000;

    while (fin.read((char*)&item, sizeof(item))) {
        if (item.code > code)
            code = item.code;
    }

    return code + 1;
}

void addItem() {
    Item item;
    item.code = generateCode();

    cout << "Enter item name: ";
    cin >> ws;
    cin.getline(item.name, 50);

    cout << "Enter unit price: ";
    cin >> item.price;

    cout << "Enter quantity: ";
    cin >> item.quantity;

    if (!cin || item.price < 0 || item.quantity < 0) {
        cout << "Invalid input.\n";
        cin.clear();
        cin.ignore(10000, '\n');
        return;
    }

    ofstream fout(FILE_NAME, ios::binary | ios::app);

    if (!fout) {
        cout << "File opening error.\n";
        return;
    }

    fout.write((char*)&item, sizeof(item));
    fout.close();

    cout << "Item added successfully.\n";
    cout << "Generated item code: " << item.code << '\n';
}

void modifyItem() {
    int code;
    cout << "Enter item code to modify: ";
    cin >> code;

    fstream file(FILE_NAME,
                 ios::binary | ios::in | ios::out);

    if (!file) {
        cout << "Item file not found.\n";
        return;
    }

    Item item;
```

```
    while (true) {
        streampos pos = file.tellg();

        if (!file.read((char*)&item, sizeof(item)))
            break;

        if (item.code == code) {
            cout << "Current name: " << item.name << '\n';
            cout << "Current price: " << item.price << '\n';
            cout << "Current quantity: "
                 << item.quantity << '\n';

            cout << "Enter new item name: ";
            cin >> ws;
            cin.getline(item.name, 50);

            cout << "Enter new unit price: ";
            cin >> item.price;

            cout << "Enter new quantity: ";
            cin >> item.quantity;

            if (!cin || item.price < 0 ||
                item.quantity < 0) {
                cout << "Invalid input.\n";
                cin.clear();
                file.close();
                return;
            }

            file.clear();
            file.seekp(pos);
            file.write((char*)&item, sizeof(item));
            file.close();

            cout << "Item modified successfully.\n";
            return;
        }
    }

    file.close();
    cout << "Item code not found.\n";
}

int main() {
    int choice;

    do {
        cout << "\n1. Add New Item";
        cout << "\n2. Modify Existing Item";
        cout << "\n3. Exit";
        cout << "\nEnter choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addItem(); break;
            case 2: modifyItem(); break;
            case 3: cout << "Exiting.\n"; break;
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 3);

    return 0;
}
```

Important Concepts for the Practical Exam

ios::binary — opens a file in binary mode.
ios::app — appends a new record to the file.
read() and **write()** — read and write binary records.
tellg() — obtains the current input position.
seekp(pos) — moves the output pointer to a record position.
sizeof(record) — gives the size of one structure record.

Save the programs separately as employee.cpp and item.cpp. Each program has its own main() function, so compile and run them separately. The data files employees.dat and items.dat are created when records are added.

```
g++ employee.cpp -o employee
g++ item.cpp -o item
```